

ozGUI / ozLib Supplement:

Multicomponent and Polydisperse Extension

Polydisperse hard-core Yukawa with the Rogers–Young closure, adhesive hard spheres, and the SASfit plugin

ozGUI/ozLib project

September 5, 2026

Abstract

This supplement extends *ozGUI / ozLib: Ornstein–Zernike Solver Toolkit* to genuinely multicomponent systems. The scalar solver is generalised to matrix-valued correlation functions, giving partial structure factors $S_{ij}(q)$ for polydisperse mixtures under *any* closure already in the library. Concretely we add: a polydisperse hard-core Yukawa potential with moment-matched size classes; the Rogers–Young closure for mixtures with its mixing parameter fixed by thermodynamic consistency rather than supplied by the user; two new GUI tabs; and a C implementation shipped as a SASfit structure-factor plugin. Every claim below is accompanied by the numerical check that supports it, and §13 documents nine defects found during validation, several of which produced plausible but wrong answers rather than obvious failures.

Contents

1	Scope and design constraint	2
2	Multicomponent Ornstein–Zernike equations	2
2.1	Matrix form	2
2.2	Implementation	3
2.3	Public interface: 1-D curves preserved	3
3	Structure-factor conventions	3
4	Discretising a continuous size distribution	4
4.1	Moment matching	4
4.2	Verification	4
4.3	How many classes are needed?	4
5	Polydisperse hard-core Yukawa	5
5.1	Charge-parameterised form	5
5.2	Size dependence of the charge	5
5.3	MSA-consistent (z, K) form	6
6	Rogers–Young for mixtures	6
6.1	Closure	6
6.2	Thermodynamic consistency fixes α	6
6.3	Validation against the literature	7
6.4	Cross-validation against the analytic MSA path	8

7 Solver strategy	8
8 The SASfit plugin	9
9 New GUI tabs	10
10 Generic polydisperse models: any potential, any closure	11
10.1 Reusing eighteen setters unmodified	11
10.2 Quadrature rule per distribution	11
10.3 Structure factor and form factor need different resolutions	12
10.4 Length scale and the radius convention	12
11 Closure suite extended and cross-validated	13
12 Consistency search for a general potential	14
13 Defects found during validation	14
14 Regression testing	17
15 Known limitations	17

1 Scope and design constraint

The starting point was that the existing solver stack is strictly one-component: $c(r)$, $h(r)$ and $\gamma(r)$ are scalar arrays of length N_r , and every closure in `update_c` acts on them elementwise.

The design constraint adopted here was that **one-component behaviour must remain bit-identical**. This is verifiable rather than aspirational: a regression suite of eight potential/closure combinations is compared against values recorded before the change (§14).

A second observation made the work far smaller than expected. Every solver class — Picard, the two Anderson variants, the two `scipy` solvers, Biggs–Andrews and both SUNDIALS KINSOL drivers — is *already* agnostic to the shape of the unknown. Searching the whole solver layer for `numberOfRadialSamplingPoints` returns no hits: each iterates a flat vector through `fixPointOperator` and never inspects what it represents. Consequently the multicomponent problem can be carried through the existing acceleration machinery unchanged, packed as the $p(p+1)/2$ unique pairs of the symmetric $\Gamma_{ij}(r)$.

2 Multicomponent Ornstein–Zernike equations

2.1 Matrix form

For p species with number densities ρ_i , the OZ equations couple $p(p+1)/2$ independent pair functions,

$$h_{ij}(r) = c_{ij}(r) + \sum_{\lambda=1}^p \rho_{\lambda} \int d\mathbf{r}_1 h_{i\lambda}(r_1) c_{\lambda j}(|\mathbf{r} - \mathbf{r}_1|). \quad (1)$$

In Fourier space this becomes an algebraic matrix relation. With $\mathbf{C}(q)$ the matrix of $\hat{c}_{ij}(q)$ and $\boldsymbol{\rho} = \text{diag}(\rho_i)$,

$$\mathbf{H}(q) = [\mathbf{I} - \mathbf{C}(q)\boldsymbol{\rho}]^{-1}\mathbf{C}(q), \quad \boldsymbol{\Gamma}(q) = \mathbf{H}(q) - \mathbf{C}(q). \quad (2)$$

The single scalar division of the one-component code, $\hat{\gamma} = \hat{c}/(1 - \rho\hat{c}) - \hat{c}$, is replaced by (2), solved as a small dense linear system at each of the N_r grid points. Nothing else in the fixpoint operator

changes: the closure step and the two Hankel transforms are the same operations applied to (p, p, N_r) arrays.

2.2 Implementation

The changes to `oZfixpointOperator.py` and `oZsolver.py` are:

- the Hankel transforms act on the *last* axis (`f.shape[-1]`, `dst(..., axis=-1)`), which is identical for 1-D input but also accepts the pair-matrix arrays;
- `isMulticomponent()`, `numberOfUniquePairs()`, `packPairs()` and `unpackPairs()` convert between the symmetric (p, p, N_r) matrix and the flat vector the solvers expect;
- `fixPointOperatorForGammaMulticomponent()` implements (2);
- `calculateSqMulticomponent()` and a multicomponent branch in `calculateRDF()` produce the observables;
- `setStartValue()` accepts the longer vector.

Remark 1. *Because every closure in `update.c` is elementwise `numpy`, `PY`, `HNC`, `RY`, `HMSA`, `Verlet`, `BPGG` and the rest all become multicomponent as soon as the potential arrays carry pair indices. Closures requiring an auxiliary reference solve (`RHNC`, `MHNC`, `RMSA`, `EuRah`) remain one-component, as do those reading `repulsivePartOfP2Ppotential` (`HMSA`, `VM`, `CJVM`, `BB`, `DH`, `CG`, `SMSA`), whose split arrays are still 1-D.*

2.3 Public interface: 1-D curves preserved

The public getters deliberately continue to return *one*-dimensional curves, so the plot tabs, `ozLib.solve()`'s curve dictionary and the ASCII/CSV/Excel/clipboard exports need no modification:

$$\text{getRDF}() \rightarrow g_{NN}(r) = \sum_{ij} x_i x_j g_{ij}(r), \quad (3)$$

$$\text{getSq}() \rightarrow S^M(q) \text{ (the measured structure factor)}. \quad (4)$$

The full matrices are exposed alongside as `partialRadialDistributionFunction` and `partialStructureFactor`, both of shape (p, p, N_r) .

3 Structure-factor conventions

Three conventions appear in this problem and confusing them is an order-of-magnitude error, so they are stated explicitly.

D'Aguzzo–Klein (number-fraction) form. Used internally by the solver, following [1]:

$$S_{ij}^{\text{DAG}}(q) = x_i \delta_{ij} + n x_i x_j \hat{h}_{ij}(q). \quad (5)$$

Ashcroft–Langreth form. Required by `PolydisperseSASBase.I_exact`, which builds $G_i = F_i \sqrt{\rho_i}$ and contracts GSG :

$$S_{ij}^{\text{AL}}(q) = \delta_{ij} + \sqrt{\rho_i \rho_j} \hat{h}_{ij}(q). \quad (6)$$

Conversion. The two differ by a purely kinematic factor,

$$S_{ij}^{\text{AL}} = S_{ij}^{\text{DAG}} / \sqrt{x_i x_j} \quad (7)$$

which is easily checked on the diagonal: $S_{ii}^{\text{DAG}}/x_i = 1 + n x_i \hat{h}_{ii} = S_{ii}^{\text{AL}}$.

Measured structure factor. What a scattering experiment sees, weighted by the form amplitudes $b_i(q)$:

$$S^M(q) = \frac{\sum_{ij} b_i b_j S_{ij}(q)}{\sum_i x_i b_i^2}, \quad b_i(q) \propto \sigma_i^2 \frac{j_1(q\sigma_i/2)}{q}. \quad (8)$$

Conversion (7) is applied in exactly one place in each module. §13 records two separate occasions on which omitting it caused a silent factor-of-nine error.

4 Discretising a continuous size distribution

4.1 Moment matching

A continuous distribution $F(\sigma)$ must be reduced to p classes. The naive choice of p equally spaced diameters spanning $\langle\sigma\rangle \pm 3s$ is what the existing Robertus code uses. Following [2] we instead impose

$$\sum_{i=1}^p x_i \sigma_i^m = \langle\sigma^m\rangle, \quad m = 0, 1, \dots, 2p-1, \quad (9)$$

i.e. $2p$ equations for the $2p$ unknowns $\{\sigma_i, x_i\}$. The Schulz distribution is a gamma distribution, so (9) is precisely Gauss–generalised-Laguerre quadrature: with $t = 1/s^2 - 1$, the nodes and weights of the weight function $x^t e^{-x}$ give $\sigma_i = x_i^{\text{node}} \langle\sigma\rangle / (t+1)$ directly. In Python this is `scipy.special.roots_genlaguerre`; in C, `gsl_integration_fixed_laguerre`.

4.2 Verification

For $s_\sigma = 0.2$ and $p = 3$ the classes are $\sigma_i = \{0.7342, 1.0532, 1.4527\}$ with $x_i = \{0.2795, 0.6304, 0.0901\}$, and the first $2p-1 = 5$ moments are reproduced to eight decimal places (Table 1).

Table 1: Moments of the discrete three-class set against the continuous Schulz distribution, $s_\sigma = 0.2$. Exact through $m = 2p-1 = 5$.

m	discrete	Schulz	
0	1.00000000	1.00000000	exact
1	1.00000000	1.00000000	exact
2	1.04000000	1.04000000	exact
3	1.12320000	1.12320000	exact
4	1.25798400	1.25798400	exact
5	1.45926144	1.45926144	exact

4.3 How many classes are needed?

[2] report that $p = 3$ is indistinguishable from $p = 5$ up to $s_\sigma = 0.3$. This was reproduced independently (Table 2): going from $p = 1$ to $p = 3$ changes S^M by 0.54, whereas $p = 3 \rightarrow 5$ changes it by 0.0015. Polydispersity is therefore a large effect that three moment-matched classes already capture.

The peak sits at $r \approx 6.2\langle\sigma\rangle$ against a mean spacing $n^{-1/3} = 5.85\langle\sigma\rangle$, as expected for a strongly correlated charged system.

Remark 2. *The moment-matching method transfers to the adhesive-sphere model, and would let the Robertus plugin use fewer classes for the same accuracy. The specific claim “ $p = 3$ suffices” does not transfer: it was established for RY–Yukawa against Monte Carlo, and sticky spheres are far more sensitive to the contact region.*

Table 2: Convergence in the number of classes. Polydisperse hard-core Yukawa, $\langle\sigma\rangle = 250 \text{ \AA}$, $Z = 200$, $n^* = 0.005$, $L_B = 7.01 \text{ \AA}$, $s_\sigma = 0.2$, RY with $\alpha = 0.5$.

	$g_{NN}^{\max}$	position	$S_{\max}^M$	pairs
$p = 1$ (monodisperse)	2.0375	6.06	2.0356	1
$p = 3$	1.7368	6.19	1.7478	6
$p = 5$	1.7340	6.19	1.7472	15
$\max g_{NN}(1) - g_{NN}(3) = 0.3583,$	$\max S^M(1) - S^M(3) = 0.5407$			
$\max g_{NN}(3) - g_{NN}(5) = 0.0115,$	$\max S^M(3) - S^M(5) = 0.0015$			

5 Polydisperse hard-core Yukawa

5.1 Charge-parameterised form

Following [2], the pair potential is

$$\beta\varphi_{ij}(r) = A_i A_j \frac{e^{-\kappa r}}{r}, \quad r > \sigma_{ij}, \quad A_i = Z_i \sqrt{L_B} \frac{e^{\kappa\sigma_i/2}}{1 + \kappa\sigma_i/2}, \quad (10)$$

with a hard core at $\sigma_{ij} = (\sigma_i + \sigma_j)/2$. The amplitude *factorises per species*, because $\exp(\kappa\sigma_{ij}) = \exp(\kappa\sigma_i/2)\exp(\kappa\sigma_j/2)$; this is what allows the $A_i A_j$ form.

Screening. κ is the inverse Debye length due to the *small ions*. For a salt-free suspension with monovalent counterions, charge neutrality gives $n_{\text{counter}} = \sum_i n_i Z_i$ and hence

$$\kappa^2 = 4\pi L_B \sum_i n_i Z_i \quad (11)$$

— the *first* power of Z . Using Z^2 produces a near-ideal-gas $S(q)$ (see §13).

5.2 Size dependence of the charge

How the valence scales with size is genuinely unsettled, so it is a user input rather than a constant. [1] assume constant surface charge density, $Z \propto \sigma^2$, and explicitly flag it as having “little precise experimental information”. Charge-renormalisation saturation (Alexander *et al.*) and electrophoresis on highly charged low-salt spheres both indicate $Z \propto \sigma$ instead. Three routes are provided:

chargeExponent	$Z_i = Z_{\text{ref}}(\sigma_i/\langle\sigma\rangle)^n$; $n = 2$ constant surface charge density (default), $n = 1$ linear in size, $n = 0$ size-independent
componentValences	explicit per-component array, overrides the power law
screeningValences	a <i>different</i> charge for κ , so screening may use the bare charge while the amplitude uses the effective one

The default $n = 2$ exists only so that published D’Aguanno–Klein setups reproduce; it is not a recommendation. Charge renormalisation is deliberately *not* implemented: its definition for a polydisperse system is unsettled (even for one component the Alexander edge-linearisation Z_{eff} and the extrapolated point charge Q_{eff} disagree), and embedding one choice inside a fitting routine would hide a contested assumption.

Magnitude of the effect. Table 3 shows the choice is comparable in size to the polydispersity effect itself, so it is not a detail. Note also that α -scaling and screening are coupled through

Table 3: Effect of the charge-scaling exponent. $p = 3$, $s_\sigma = 0.2$, D’Aguanno–Klein parameters, RY $\alpha = 0.5$.

scaling	Z_i	κ	$g_{NN}^{\max}$	$S_{\max}^M$
$n = 2$ (default)	107.8 / 221.8 / 422.0	0.6054	1.7368	1.7478
$n = 1$	146.8 / 210.6 / 290.5	0.5936	1.9262	1.8381
$n = 0$	200 / 200 / 200	0.5936	2.0384	1.7724

(11): changing the exponent moves both the amplitude prefactor and the screening length unless **screeningValences** decouples them.

An internal consistency check falls out of (11): $n = 0$ and $n = 1$ must share a κ , because $\sum_i x_i \sigma_i = \langle \sigma \rangle = 1$ by construction, while $n = 2$ raises it by exactly $\sqrt{\langle \sigma^2 \rangle} = \sqrt{1.04} = 1.0198$. Both hold to four decimals.

5.3 MSA-consistent (z, K) form

For direct comparison with the analytic MSA/RMSA path the same convention must be used. From the polydisperse Yukawa specification,

$$c_{ij}(r) = K_{ij} \frac{e^{-z(r-\sigma_{ij})}}{r}, \quad r > \sigma_{ij}, \quad K_{ij} = K \delta_i \delta_j, \quad \delta_i = d_i e^{-z\sigma_i/2}, \quad (12)$$

with $g_{ij}(r) = 0$ for $r \leq \sigma_{ij}$. MSA sets $c = -\beta U$, so

$$\beta U_{ij}(r) = -K \delta_i \delta_j \frac{e^{-z(r-\sigma_{ij})}}{r}, \quad (13)$$

i.e. $K > 0$ is **attractive** and $K < 0$ **repulsive**, and there is *no* σ_{ij} prefactor. Note that `PolydisperseOneYukawaMSA` takes δ_i *directly* — its default `np.ones(N)` means $\delta_i = 1$, not $d_i = 1$ — so the exponential in (12) must not be applied a second time (§13, item 9).

6 Rogers–Young for mixtures

6.1 Closure

The RY closure is applied pairwise with a *single shared* mixing function — there is no ij subscript on f in [1]:

$$h_{ij}(r) = -1 + e^{-\beta \varphi_{ij}(r)} \left[1 + \frac{e^{f(r)\gamma_{ij}(r)} - 1}{f(r)} \right], \quad f(r) = 1 - e^{-\alpha r}, \quad (14)$$

interpolating between PY ($\alpha \rightarrow 0$) and HNC ($\alpha \rightarrow \infty$).

6.2 Thermodynamic consistency fixes α

α is not a free parameter: it is determined by requiring the compressibility and virial routes to the pressure to agree. For a mixture this is a *scalar* condition — which sidesteps the apparent difficulty that a mixture has a full compressibility *matrix*:

$$\chi_{\text{comp}}^{-1} = 1 - n \sum_{ij} x_i x_j \hat{c}_{ij}(0), \quad (15)$$

$$\chi_{\text{vir}}^{-1} = \frac{\partial(\beta P)}{\partial n}, \quad (16)$$

with the virial pressure

$$\frac{\beta P}{n} = 1 + \frac{2\pi}{3}n \sum_{ij} x_i x_j \sigma_{ij}^3 g_{ij}(\sigma_{ij}^+) - \frac{2\pi}{3}n \sum_{ij} x_i x_j \int r^3 g_{ij}(r) (\beta U_{ij})'(r) dr. \quad (17)$$

The contact term. [1]’s equation for βP omits the second term of (17), legitimately, because their macroions are so strongly charged that $g_{ij}(\sigma_{ij}^+) \approx 0$. That is *not* safe in general: this model has a genuine hard core and K may be small, so contact is reachable and the term is first order in the pressure. It is included here, with g_{ij} at contact obtained by linear extrapolation from the first two grid points outside σ_{ij} . This extrapolation is the least certain step in the whole search and is flagged as such in the code.

Numerics. Two traps, both encountered:

1. the **pair potential must be held fixed** while n is perturbed. In the charge-parameterised form (10) κ depends on the counterion density, so rebuilding the system at $n \pm \Delta n$ differentiates a density-dependent potential; the symptom is a residual stuck near +65 that never crosses zero. In the (z, K) form (12) the potential is density-independent and the problem does not arise.
2. **cold-start each α .** Warm-starting across qualitatively different α destabilises the search; warm-starting across the $n \pm \Delta n$ perturbations is fine, and is what [1] do, holding α fixed there because it varies slowly with n .

Result. At $\varphi = 0.2$, $K = 0$, $z = 2$ with three classes the search returns $\alpha = 0.2166$ with residual 1.2×10^{-5} — a clean root, and a plausible hard-sphere value. The residual is monotone in α (Table 4).

Table 4: Thermodynamic inconsistency $\chi_{\text{comp}}^{-1} - \chi_{\text{vir}}^{-1}$ versus α ; $\varphi = 0.2$, $z = 2$, $p = 3$.

α	0.20	0.50	1.00	2.00
$K = 0$	+0.0127	−0.1847	−0.4002	−0.6054
$K = 0.5$	−0.2219	−0.3012	−0.3881	−0.4697

Remark 3 (A consistent α need not exist). *For $K = 0.5$ the residual is negative across the whole bracket: no α makes the two routes agree. The implementation returns the closest endpoint and reports “no consistent alpha found” rather than presenting it as a solution. This is a property of the closure, not a numerical failure.*

6.3 Validation against the literature

The one-component limit reproduces Table 1 of [1] (their test 3: $\beta\varphi(x) = (1000/x)e^{-0.95x}$, $n\sigma^3 = 0.01432$). The self-consistent $\alpha = 0.3942$ gives Table 5.

Table 5: One-component RY against [1] Table 1, test 3.

quantity	this work	D’Aguzzo–Klein	deviation
$1/S(0)$	93.787	93.80	0.01%
P^{ex}	36.458	36.45	0.02%
U^{ex}	20.145	20.09	0.27%
g_{max}	2.069	2.08	0.54%

The two quantities that *enter* the consistency condition agree to 0.01–0.02%, which is the sharpest available test. Two further checks:

- **HNC limit.** At large α the solver gives $g_{\max} = 1.937$, $1/S(0) = 77.69$, $U^{ex} = 20.56$, $P^{ex} = 36.94$ against the paper’s HNC column 1.94, 77.5, 20.57, 36.95 — three-digit agreement, an independent check on the closure and the transforms.
- **Grid insensitivity.** Results are flat to four decimal places from 50 to 400 points per σ ; only $1/S(0)$ shifts slightly with $r_{\max}$ through the $q \rightarrow 0$ extrapolation. Neither paper specifies its grid, and the residual differences in $g_{\max}/U^{ex}$ are therefore attributable to their (unreported) α or Δn rather than to discretisation.

6.4 Cross-validation against the analytic MSA path

Because RY and MSA both set $c = -\beta U$ outside the core, the *derivative* dS/dK at $K \rightarrow 0$ must agree even though the closures differ. This is the decisive test of the potential convention of §5.3, and it is what exposed the δ_i double-counting bug. Table 6 shows agreement after the fix.

Table 6: dS_{NN}/dK at $Q = 0.5$, $z = 2$, three classes. Agreement is near-exact where both closures reduce to the same physics and degrades with density where PY and RY genuinely differ.

φ	MSA	RY	ratio
0.002	0.03031	0.03085	1.018
0.020	0.24025	0.24900	1.036
0.100	0.41790	0.47266	1.131

Before the fix the ratio was 0.14 at all three densities, consistent with $1/e^{-z\sigma/2} = 1/0.135 = 7.4$.

7 Solver strategy

Three tiers are tried in order, fastest first, each strictly more robust than the previous one.

Tier 1 — Anderson (KIN_FP) with Picard pre-conditioning. Fastest where it works. Anderson diverges from a cold start on this problem: a hand-written implementation was *slower* than undamped Picard when damped enough to be stable (851 versus 677 iterations) and diverged otherwise, and SUNDIALS’ KIN_FP diverged too, damped or not. The remedy is a short run of plain undamped Picard — which *is* contractive here — before handing over. Measured: 0 pre-steps diverges, 20 already works, 40 is the time optimum; 50 is the default for margin.

Tier 2 — Newton–Krylov (GMRES with KIN_LINESEARCH). A different algorithm, not a variant: damped Newton steps on the residual $F(u) = G(u) - u$ with the Jacobian applied matrix-free through GMRES. It converged in *every* regime tried, in a strikingly constant 31–40 Newton iterations regardless of difficulty — the signature of global convergence via linesearch. The cost is wall-clock: each Newton step requires many GMRES inner iterations, each a full function evaluation, so it runs 3–6 $\times$ slower than Anderson where Anderson works.

Tuning follows SASfit’s own `KIN_sasfit_configure()`: `MaxNewtonStep= 100n` (not n), `FuncNormTol=  $10^{-10}$` and `ScaledStepTol=  $10^{-13}$` (deliberately different from each other), `MaxRestarts= 10`, `ETACHOICE1`.

Tier 3 — Picard , undamped then progressively damped.

The Python side. `sundials4py` is available as a prebuilt wheel on PyPI (version 7.8.0), so no binding generation or C++ build is needed. Measured on the same problem, `KIN_FP` is the best of the Python solvers on every count: it agrees with the others where they converge, is 2–4× faster, and on the hard state point it fails to converge in 0.9 s rather than converging onto a spurious root.

state point	KIN_FP	scipy Anderson	Picard
$\varphi = 0.02, K = 1$	0.1 s	0.2 s	0.1 s
$\varphi = 0.1, K = 0.2$	0.2 s	0.8 s	—
$\varphi = 0.1, K = 1$	rejected, 0.9 s	rejected, 5.6 s	diverged, 56 s

A caution recorded earlier in this project — that `sundials4py` might silently ignore `KINSetMAA` when it is called after `KINInit`, as `pyozGUI` does — was *tested and found to be unfounded*. On a deliberately slow linear fixed point, `MAA = 0` took 757 iterations and `MAA = 5` took 93, *identical in both call orders*, so Anderson acceleration is genuinely active. The C requirement is nonetheless real: calling `KINSetMAA` after `KINInit` segfaults on SUNDIALS 7.1.1, so the C core keeps the documented order.

Table 7: Solver tier actually used, and wall-clock time, across seven parameter regimes (C implementation, SUNDIALS 7, $N_r = 4095$).

regime	time	tier
$p = 3, s = 0.2$	0.25 s	Anderson
$p = 1$ monodisperse	0.03 s	Anderson
$p = 5, s = 0.3$	6.85 s	Newton–Krylov
$p = 3, s = 0.4$	0.32 s	Anderson
$p = 3, s = 0.2, 5\times$ density	0.29 s	Anderson
$p = 3, s = 0.2, 0.2\times$ density	0.25 s	Anderson
$p = 7, s = 0.25$	4.11 s	Anderson

Never trust the convergence flag. `KINsol` can return `KIN_SUCCESS` after diverging: once the RY closure’s exponent clipping pins values near 10^{217} the internal stopping test is defeated. This was observed at $p = 5, s_\sigma = 0.3$, where S^M was returned as $\sim 10^{222}$ *with a success flag*. Both `KINsol` drivers therefore recompute the residual after `KINsol` returns and reject the result unless it really is small. With that check in place the regime correctly falls through to Newton–Krylov and returns the right answer.

8 The SASfit plugin

The C implementation is shipped as `src/plugins/ry_polydisperse_yukawa/`, exporting `sasfit_sq_RYPolydisperseYukawa`. It follows the established plugin conventions: `sasfit_cmake_plugin()`, `include/private.h`, Doxygen `\defgroup/\ingroup` with an HTML parameter table, `SASFIT_PLUGIN_EXP_*` registration, and the `func/_f/_v` triad with `_f` and `_v` stubbed to 0 as in SASfit’s own `sq_*` plugins.

Transform. The radial transform is a DST-I built from a radix-2 real FFT, which is why $N_r + 1$ must be a power of two; $N_r = 2^{12} - 1$ at 100 points per σ matches the Python grid exactly. The DST-I agrees with `scipy.fft.dst(type=1)` exactly.

Caching. A solve costs a few tenths of a second, so re-solving per q -point would make any scan or fit unusable. The solved system is cached on the full parameter set and successive solves

warm-start from the previous γ ; 2000 further q -points after the first cost effectively nothing.

Portability. Builds and runs against both SUNDIALS 7.1.1 and 6.4.1 with identical results (v7 about 30% faster). The one relevant API break is `SUNContext_Create`, whose first argument became a typed `SUNComm` with a `SUN_COMM_NULL` sentinel in v7; this is handled by a macro switched on `SUNDIALS_VERSION_MAJOR`. Version 7 also requires `-lsundials_core`, which does not exist in v6.

Performance history. Four measured steps, each kept as a test:

change	time
initial (GSL LU_decomp per q -point, damped Picard)	15.1 s
inline small-matrix solve + upper-triangle symmetry	8.3 s
undamped Picard	2.3 s
Anderson (KIN_FP) + Picard pre-conditioning	0.26 s

The first version was *slower than `numpy`*, because it performed 3.4 million GSL 3×3 LU_decomp calls, where GSL's generality dominates the actual arithmetic.

Agreement with Python. The full $S^M(q)$ curve matches the Python implementation to 7.1×10^{-11} relative on the Picard path and 1.3×10^{-8} on the Anderson path (the latter being the solver tolerance, not an error).

α in the plugin. The consistency search costs three extra OZ solves per trial value and is far too slow inside a fit. The plugin therefore takes α as a parameter; it should be determined once in the GUI (§6.2) and then held fixed, which is justified because α varies slowly with density — the same reasoning [1] use when holding it constant across their own finite-difference density derivative.

9 New GUI tabs

`ozgui.py` gained a top-level notebook and an `_addExtraTabs()` hook driven by a registry,

```
EXTRA_TABS = [
    ("polydisperse_yukawa_tab", "PolydisperseYukawaTab", "Polydisperse Yukawa"),
    ("robertus_shs_tab", "RobertusSHSTab", "Robertus SHS"),
    ("ry_polydisperse_yukawa_tab", "RYPolydisperseYukawaTab", "RY Polydisperse Yukawa"),
]
```

Each tab is a self-contained `ttk.Frame` isolated by `try/except`, so adding one is a single line and an import failure costs only that tab.

Both new tabs follow the same pattern: a parameter panel; plot tabs for $I(Q)$, the approximation error, $S_{ij}(Q)$, $S(Q)$ and the size distribution; a Summary tab; a threaded compute handing results back through a queue (Tk is not thread-safe); and ASCII/CSV export.

Approximation schemes. Because both new SAS classes derive from the same `PolydisperseSASBase`, all six of SASfit's schemes for combining a structure factor with a size distribution — monodisperse, decoupling (Kotlarchyk–Chen), local monodisperse (Pedersen), partial structure factors, scaling (Gazzillo) and van der Waals one-fluid — come for free, and the error of each can be read directly off the plot. The approximation code is byte-identical between the tabs, so comparing them is a genuine test of how those approximations fare for a different interaction rather than a comparison of two implementations.

Failure reporting. An individual scheme that fails does not lose the run: it is marked “unavailable” in the summary. This matters because the *monodisperse* reference can have no PY solution at a state point where the polydisperse system does. Adhesive-sphere failures are additionally translated into actionable advice — *increase τ* , since smaller τ is stickier and below $\tau_c(\varphi)$ the adhesive PY closure has no physical solution at all.

α is not exposed. Following §6.2, the RY tab has no α entry field. It is solved for on every Compute and reported in the Summary, together with the residual and an explicit warning when no consistent value exists.

10 Generic polydisperse models: any potential, any closure

The multicomponent machinery of §3 was initially reached only through one bespoke potential. This section describes its generalisation to *any* of the library’s eighteen one-component potentials, combined with any closure that works multicomponent and any form factor.

10.1 Reusing eighteen setters unmodified

The potentials are not rewritten. `getrArray()` gains an override; the builder points it at r/σ_{ij} and calls the ordinary one-component setter. Because every setter measures its hard core against `hardSphereDiameter`, which is unity in those reduced units, that core lands exactly at σ_{ij} while the tail is evaluated at the reduced separation. The mixing rule is therefore

$$\sigma_{ij} = \frac{1}{2}(\sigma_i + \sigma_j), \quad u_{ij}(r) = u(r/\sigma_{ij}), \quad (18)$$

i.e. every pair sees the same interaction *shape* in units of its own contact distance. This is the assumption Robertus *et al.* make for size-independent stickiness, and it is what makes the construction potential-agnostic: ε , δ , n , τ keep their reduced-unit meaning for every pair, so no per-potential mixing rule need be invented.

Remark 4. Equation (18) is a modelling choice, not a derivation. For Lennard-Jones the conventional alternative is Lorentz–Berthelot, $\varepsilon_{ij} = \sqrt{\varepsilon_i \varepsilon_j}$ with per-species ε ; that is a different model and would require per-species tail parameters, which the builder deliberately does not invent.

Charge-coupled potentials are refused. DLVO, DLVO-Hydra, IonicMicrogel and the dedicated polydisperse Yukawa have amplitudes that scale with particle size and a κ set by the whole distribution through the counterion density (11). The reduced-tail rule is simply wrong for them, so they are rejected with a message rather than silently mis-modelled.

Verification. With $s_\sigma = 0$ and one class the generic path reproduces the ordinary one-component setters **bit-identically** (maximum difference 0.00×10^0 for hard spheres, square well and Lennard-Jones; the first and third reproduce the recorded values 2.3561180274 and 2.1635946756). This is the sharpest available check that the generalisation leaves validated physics untouched.

10.2 Quadrature rule per distribution

All four supported distributions (Schulz, Gaussian, log-normal, Weibull) have *analytic* moments, so computing $\langle \sigma^n \rangle$ is never the difficulty. The difficulty is the map from moments to nodes: the Golub–Welsch Hankel eigenproblem is classically ill-conditioned even with exact input.

A closed-form classical rule is therefore used wherever one exists, bypassing the Hankel step entirely: generalised Gauss–Laguerre for the Schulz/gamma weight and Gauss–Hermite for the

Table 8: Condition number of the log-normal Hankel matrix built from exact moments. Double precision carries about 10^{16} ; past that Cholesky does *not* fail, it silently returns nonsense.

	$s = 0.3$	$s = 0.4$	$s = 0.5$
$N = 6$	1.6×10^7	1.7×10^7	1.2×10^8
$N = 8$	2.1×10^{10}	1.9×10^{11}	4.7×10^{13}
$N = 10$	7.0×10^{13}	1.7×10^{16}	5.4×10^{19}
$N = 12$	6.4×10^{17}	1.2×10^{21}	2.3×10^{27}

Gaussian, both stable in double precision; Golub–Welsch with 200-digit arithmetic only for log-normal and Weibull, which have no classical rule. All four are moment-exact to $\sim 10^{-16}$.

A physical guard on the Gaussian. Gauss–Hermite places its outermost node at $u \approx \pm 2.86$, so σ reaches zero near $s \approx 0.35$. A form factor tolerates a vanishing bin; a structure factor does not, because a hard core must be placed at every σ_{ij} . Nodes below $10^{-3}\langle\sigma\rangle$ are dropped and the weights renormalised — sacrificing moment-exactness rather than physicality. The test is deliberately *relative*: at $s = 0.35$ the node sits at 10^{-4} , positive but still a zero-diameter particle, so a bare $\sigma > 0$ check is insufficient.

10.3 Structure factor and form factor need different resolutions

The two averages behave differently in σ . $S_{ij}(Q)$ varies smoothly with size, so three to five moment-matched classes suffice — and the solve costs $O(p^2)$ pair transforms, so p should stay small. The form-factor average $\langle|F|^2\rangle$ oscillates: at high Q the phase QR spans about $Q\langle\sigma\rangle s_\sigma$ radians across the distribution. The two class counts are therefore separated, with the rule of thumb $p_{\text{FF}} \gtrsim Q_{\text{max}}\langle\sigma\rangle s_\sigma$.

The underlying defect was subtler than resolution, and is instructive. Since $S_{ij}^{\text{AL}} = \delta_{ij} + \sqrt{\rho_i \rho_j} h_{ij}$, interpolating the *whole* matrix onto a finer size grid smears the Kronecker delta into a band — which converts the incoherent sum $\sum_i |F_i|^2$ into the coherent $|\sum_i F_i|^2$, and coherent addition of form factors is precisely what oscillates. Only h_{ij} is interpolated; the delta is re-imposed exactly.

Table 9: High- Q ripple against brute-force integration of $\langle|F|^2\rangle$ with 4000 points, $s_\sigma = 0.3$.

	ripple
brute-force reference	0.038
5 classes (before)	1.91
$p_{\text{FF}} = 40$	0.041
$p_{\text{FF}} = 160$	0.041

Remark 5 (Two plausible wrong diagnoses). *Both are recorded because each looked convincing. First, “too few classes”: disproved by its own test, since the ripple did not converge away even at 30 classes. Second, “Gauss nodes are unsuited to an oscillatory integrand”: a true statement, and replacing them with a dense grid was the right change, but not the cause — the ripple actually got worse ($1.91 \rightarrow 3.14$). The brute-force reference is what localised the fault, by showing I_{dilute} correct at 0.040 while I_{exact} gave 2.79 at $\varphi = 10^{-6}$, where S must be the identity. Ten lines of direct integration should have been written before any quadrature was tuned.*

10.4 Length scale and the radius convention

The Ornstein–Zernike equations are *always* solved in reduced units (mean diameter unity): the radial grid spans only about 41 diameters, so a physical $\sigma = 100$ would place every hard core off

the end of it. A mean radius supplied by the user is applied *after* the solve, to the diameters, the number densities ($n \propto L^{-3}$, which leaves φ invariant) and the q axis ($q_{\text{red}} = QL$); the tail parameters are already reduced and are untouched.

The scattering radius equals the hard-core radius, $R_i = \sigma_i/2$, for every class. Verified at $R = 50$: the $S(Q)$ peak moves to $Q = 0.06608$ while $Q\sigma = 6.608$ (hard spheres expect $\approx 2\pi$) and $QR = 4.496$ at the first $I(Q)$ minimum (exact sphere zero 4.493) — both unchanged from the dimensionless case, confirming a pure change of units.

Remark 6 (Known limitation). *Scattering size and interaction size are perfectly correlated by construction: no independent width, no offset. That is correct for charged colloids or bare silica, where the particle is the hard core; it is wrong for sterically stabilised particles whose brush is contrast-matched ($R < \sigma/2$), for charged colloids at low salt where the effective core exceeds the physical particle, and for solvated or partially matched shells.*

11 Closure suite extended and cross-validated

The closure suite was checked against `OrnsteinZernike.jl`, an independently written Julia library published with a peer-reviewed review. Bridge functions were compared at prescribed γ by evaluating $B = \ln g + \beta u - \gamma$ from this library's $c(r)$.

Table 10: Bridge functions against the independent Julia implementation.

closure	$\max B_{\text{ours}} - B_{\text{Julia}} $
Verlet	1.1×10^{-16}
BPGG	1.1×10^{-16}
Bomont–Bretonnet	3.3×10^{-16}
Martynov–Sarkisov	2.2×10^{-16}
Charpentier–Jackse (CJVM)	identical formula
Zerah–Hansen (HMSA)	identical after substitution

Choudhury–Ghosh differs by parameterisation rather than error: this library uses a density-dependent coefficient $1.0175 - 0.275 \cdot 6\varphi/\pi$ whose zero-density limit is exactly Julia's constant $\alpha = 1.01752$.

Remark 7. *A first run reported Verlet as differing. The cause was mine: Julia's default is $B = 8/5$ and it writes the denominator as $1 + B\gamma/2$, which is the same 0.8γ used here. Checking the actual defaults before reporting a discrepancy is the lesson; the same pattern — an apparent disagreement that is really an assumption of one's own — recurs throughout this project.*

Three closures were added from that comparison: Khanpour ($B = \ln(1 + \alpha\gamma)/\alpha - \gamma$), Modified Verlet, and Extended Rogers–Young, whose bridge function is the ordinary RY construction with one extra quadratic term,

$$\phi = \frac{e^{f\gamma} - 1}{f}, \quad B = -\gamma + \ln(1 + \phi + a\phi^2), \quad (19)$$

so that $a = 0$ reduces *exactly* to RY — verified to 2.5×10^{-16} , an internal check independent of the Julia transcription. All three inherit the multicomponent path, every solver and the GUI dropdown automatically, because each is an elementwise expression in γ .

Remark 8 (Naming). *This library's KH is Kovalenko–Hirata, which abbreviates identically to Khanpour but is a different closure; the new one is therefore spelled out in full. CJVM was confirmed to be Charpentier–Jackse, its formula matching Julia's exactly.*

Availability by potential. Closures needing a one-component reference solve (Reference HNC, Modified HNC) or a hard-sphere-specific construction (Rescaled MSA, EuRah, ZSEP) cannot be used with a polydisperse potential, and the GUI now offers 19 of 23 rather than offering all and failing. Martynov–Sarkisov, Vompe–Martynov and CJVM are deliberately *not* hidden: they are structurally fine and fail only because their square-root bridge overflows $\exp(\gamma + B)$ for strongly coupled charged systems, a limitation that applies equally in one component.

12 Consistency search for a general potential

The scalar condition (15)–(17) is available for any potential built by the generic route. Two points differ from §6.2. First, $(\beta U)'$ is differentiated *numerically*, because the builder reuses arbitrary setters and cannot know their analytic derivative. Second, the reduced-tail potential (18) is density-independent, so perturbing φ leaves it untouched — which is not automatic, and is exactly the trap that produced a residual stuck near +65 in the charge-parameterised case.

Table 11: Both routes against analytic Percus–Yevick hard spheres, $\eta = 0.3$. Residual differences are grid discretisation.

quantity	analytic	measured
χ^{-1}	10.6622	11.0678
$\beta P/\rho$	3.8163	3.8896
PY residual	+1.2482	+1.3284

The Rogers–Young residual crosses zero between $\alpha = 0.1$ and 0.5 and converges onto the HNC limit at large α (−4.1495 against HNC −4.1497) — exactly the behaviour that makes the closure work. A live search returns $\alpha = 0.2441$ with residual -6.9×10^{-5} .

Remark 9 (Judge the residual relatively). χ^{-1} is of order 10 for a dense fluid, so an absolute residual of 10^{-3} is a relative 10^{-4} . An absolute threshold rejected a perfectly good root ($\alpha = 0.2443$, residual -1.2×10^{-3}) as “no consistent value found”. The criterion is now relative to χ^{-1} .

Remark 10 (A false alarm worth recording). The search was first exercised at $s_\sigma = 0.2$, $p = 3$, $\varphi = 0.3$, where the residual saturated near +2.11 with almost no α dependence, and this was read as a broken virial route. With three moment-matched classes the largest diameter reaches $\sigma \approx 2.48$, so at $\varphi = 0.3$ the biggest spheres are at or past their own close packing: the state was unreachable, not the code wrong. Diagnosing a numerical defect from a single pathological point, without first checking a case with a known answer, was the error.

13 Defects found during validation

Each of the following produced plausible-looking wrong answers rather than an obvious failure, which is why they are recorded.

1. **HMSA returned $S(q) = 1$ everywhere** for hard spheres and for every potential except Lennard-Jones, Yukawa and DLVO. The cause: `repulsivePartOfP2Ppotential` and `attractivePartOfP2Ppotential` were never populated for those potentials, so $e^{-0} = 1$ gave no hard-core exclusion and $c(r) = 0$ became an exact fixed point. Fixed by a general fallback in `setPotentialByName()`. Independently re-confirmed afterwards: HMSA and RY are now bit-identical for hard spheres, as the physics requires (no attractive tail).
2. **Screening used Z^2 instead of Z** . Equation (11) involves the first power. The wrong version gives $g_{\max} \approx 1.006$, i.e. an essentially ideal gas.

3. **The virial route differentiated a density-dependent potential** (§6.2).
4. **KINSol reported success after diverging** (§7).
5. **KINSetMAA must precede KINInit**. The Python binding calls them in the opposite order; doing so in C segfaults on SUNDIALS 7.1.1, because the Anderson depth is set without the corresponding arrays being allocated. That Python tolerates it suggests the setting may simply be ignored there — in which case that path runs plain fixed-point iteration rather than the Anderson acceleration it appears to request. This is worth checking on the Python side.
6. **DST-I factor of two**. For an odd extension, $\text{Im } V[k+1] = -y[k]$; the factor 2 is already in the DFT.
7. **Warm starting silently did nothing**, because `ryp_build_potential` cleared the stored guess — precisely the case in which it should be kept.
8. **A loop that exhausted its iteration budget returned success**, because the status variable was overwritten by each call to the fixpoint map.
9. **Two placeholder “engines” that computed nothing**. Both the adhesive-sphere and the Rogers–Young Python cores shipped as stubs that returned plausible numbers without solving anything.

`robertus_shs_core_py.py`’s `solve()` body was a bare `pass` under a docstring reading “Dummy solve method for compatibility” — the multicomponent PY λ_{ij} system was never formed. `S_matrix()` started from `np.ones((p,p))` and filled *every* element, diagonal and off-diagonal alike, from a *single*-component expression $1/(1+24\varphi A)$ times an ad-hoc “sticky correction”, clamped with `np.maximum(S,0.1)` to force positivity. The size classes were therefore never coupled.

The diagnostic that exposed it is worth remembering: in the dilute limit an Ashcroft–Langreth S must tend to the *identity*. The placeholder gave diagonal ≈ 1 and *off-diagonal* ≈ 1 , i.e. an all-ones matrix. I_{exact} then collapses from $\sum_i \rho_i F_i^2$ to $(\sum_i \sqrt{\rho_i} F_i)^2$, inflating it by $(\sum_i \sqrt{w_i})^2$ — a factor depending only on the *number of classes*. Measured $I_{\text{exact}}/I_{\text{monodisperse}}$ at $s_\sigma = 0.01$, where every scheme must agree, was 1.00, 1.97, 2.53, 5.69, 8.84, 18.2 for 1, 2, 4, 8, 12, 24 classes, and the dilute limit gave 8.83 instead of 1.

This also explains the reported symptom that “the exact model does not respond to the distribution width”: polydispersity entered only through the class weights in that spurious sum, never through the physics. The approximation schemes call the real `PolydisperseSASBase` machinery and so did respond — hence exact and approximate disagreeing in both scale and width dependence. Note the first diagnosis attributed this to a missing Ashcroft–Langreth conversion; that was wrong, since the diagonal was already correct, and only the off-diagonal test identified the true cause.

Replaced by a faithful port of the C engine (`py_compute_ab`, `py_residual`, `build_Q_matrix`, `rshs_structure_matrix`). Afterwards: dilute off-diagonal 2×10^{-7} , $I_{\text{exact}}/I_{\text{dilute}} = 0.99999$, $I_{\text{exact}}/I_{\text{monodisperse}} = 1.000$ for *every* class count, λ residual 2×10^{-16} , and $S(q)$ varying smoothly with width (0.7276, 0.7481, 0.8050, 0.8760 at $s_\sigma = 0.01, 0.1, 0.2, 0.3$).

Separately, `ry_polydisperse_core_py.py` had no density or volume fraction at all in its constructor, performed no OZ iteration, and built S_{ij} from an ad-hoc closed-form blend. It was replaced by an adapter onto the real multicomponent solver (§5.3).

10. **Unconverged and spurious solutions returned as answers.**

`picardIteration()` prints “Picard did not converge after N steps” and then calls `derivePhysicalQuantitiesFromFixpoint()` regardless, exposing no flag, so callers cannot tell. For an attractive Yukawa ($K = 1$, $\varphi = 0.1$, $z = 2$) it returned a plausible $S(Q) = 2.65$ after diverging to a residual of 4.8×10^{204} .

Adding a residual check is necessary but *not sufficient*: the RY equations have multiple fixed points, and different solvers converge cleanly onto different ones. At that same state point the hand-written Anderson gave $S(0.5) = 17.03$ and `scipy` Anderson gave 9.71, *both* with residual $\sim 10^{-12}$ — and with $\min S_{NN}(q) = -9.19$ and -17.21 . A structure factor is a variance and cannot be negative, so both were spurious roots. A physicality screen ($\min S_{NN} \geq 0$) is therefore applied in addition to the residual check. Biggs–Andrews makes the same point from the other side: it declared convergence after 21 steps and failed the residual check.

11. **The Kronecker delta must not be interpolated.** Smearing δ_{ij} across a finer size grid turns the incoherent $\sum_i |F_i|^2$ into the coherent $|\sum_i F_i|^2$, reinstating exactly the form-factor oscillations that polydispersity should remove (§10.3). It presented as a physics question — “at $s_\sigma = 0.3$ no oscillations should be visible” — and was correct as such.
12. **A single class stored as a $(1, 1, N)$ array.** `isMulticomponent()` is false either way, so the scalar code path ran on three-dimensional input and every derived curve came back shaped $(1, 1, N)$. `getRDF().max()` still returned the right number, which is precisely why the bit-identical $p = 1$ check passed and hid it; anything expecting a *curve* then failed with “object too deep for desired array”.
13. **A convergence test against the wrong array.** `x_0` holds the *start* value, not the converged fixpoint, so every monodisperse reference solve was rejected. It surfaced as all six approximation schemes reporting “no monodisperse MSA solution” — a message from the base class that looked like a physical obstruction rather than a caller’s bug.
14. **Two silent rejections in `setPotentialByName()`.** It counted arguments *with defaults* as mandatory, making any setter that carries optional parameters unreachable; and array-valued parameters appeared in the GUI as numeric boxes defaulting to 1.0, failing a length check. Both returned quietly after printing, leaving the potential untouched so that every closure produced the trivial $S(q) = 1$. The array parameters are now keyword-only, so `getfullargspec()[0]` — which the GUI builds its fields from — lists only the numbers a user can type.
15. **GUI defaults contradicting the documented physics.** Every potential parameter box was seeded with 1.0, so a user who left `chargeExponent` alone silently got $n = 1$ (charge linear in size) while the documentation promised $n = 2$ (constant surface charge density) — a different model, with nothing on screen to say so. Fields are now seeded from each setter’s own defaults.
16. **Tk touched from a worker thread.** Reading the solver dropdown inside the compute worker raised “main thread is not in main loop” and lost the entire run. Such controls are now read on the main thread and carried into the worker.
17. **Two convention errors, each worth about an order of magnitude.** (a) The Ashcroft–Langreth conversion (7) was omitted in the adhesive-sphere SAS layer, inflating I_{exact} by a factor that scales with the number of classes: the ratio $I_{\text{exact}}/I_{\text{monodisperse}}$ at $s_\sigma = 0.01$, where every scheme must agree, was 1.00, 1.97, 2.53, 5.69, 8.84 and 18.2 for 1, 2, 4, 8, 12, 24 classes, and the dilute limit gave 8.83 instead of 1. Because the number of classes is held fixed while sweeping the width, the exact curve sat about nine times too high and its genuine width dependence was visually swamped — the

reported symptom was “the exact model looks independent of the distribution width”.
 (b) In the RY wrapper, $\exp(-z\sigma_i/2)$ was applied to `delta`, double-counting it because `PolydisperseOneYukawaMSA` takes δ_i directly. RY’s K -response came out a factor 7.2 too weak. Notably the *first* implementation was correct and a subsequent “correction” made from reading the specification introduced the error: only the numerical cross-check of Table 6 settled it.

14 Regression testing

The one-component regression suite covers eight combinations: PY, HNC, RY, HMSA and MSA on hard spheres; PY and RY on Yukawa; HNC on Lennard-Jones. Each records $g_{\max}$, $\sum g$, $c(0)$, $\sum c$, $S_{\max}$ and $S(0)$. After the multicomponent extension all eight are **bit-identical** to the pre-change baseline. The suite is re-run against the deployed files rather than a working copy, and should be re-run after any change to the fixpoint operator.

15 Known limitations

1. Closures requiring an auxiliary reference solve (RHNC, MHNC, RMSA, EuRah) remain one-component.
2. Closures reading the repulsive/attractive potential split (HMSA, VM, CJVM, BB, DH, CG, SMSA) are not yet wired for the multicomponent potential; those arrays remain 1-D. PY, HNC, RY, Verlet and BPGG work. RY is the one that matters here.
3. `ryp_SM()` interpolates linearly between q -grid points, which underestimates the main peak by about 0.05%.
4. The plugin’s cache is a single static and therefore not thread-safe. SASfit evaluates a model serially in q , which matches how it is driven.
5. The contact-value extrapolation in (17) is the least certain step of the consistency search.
6. A thermodynamically consistent α does not exist everywhere in parameter space; this is reported rather than hidden.
7. Charges are effective (renormalised), not bare (§5.2).

References

- [1] B. D’Aguanno and R. Klein, *Structural effects of polydispersity in charged colloidal dispersions*, J. Chem. Soc. Faraday Trans. **87**, 379 (1991).
- [2] B. D’Aguanno and R. Klein, *Integral-equation theory of polydisperse Yukawa systems*, Phys. Rev. A **46**, 7652 (1992).
- [3] F. J. Rogers and D. A. Young, *New, thermodynamically consistent, integral equation for simple fluids*, Phys. Rev. A **30**, 999 (1984).
- [4] C. Robertus, A. P. Philipse, J. G. H. Joosten and Y. K. Levine, *Solution of the Percus–Yevick approximation of the multicomponent adhesive spheres system applied to the small angle X-ray scattering from microemulsions*, J. Chem. Phys. **90**, 4482 (1989).
- [5] L. Blum and J. S. Høye, *Solution of the Ornstein–Zernike equation with Yukawa closure for a mixture*, J. Stat. Phys. **19**, 317 (1978).
- [6] L. Blum and M. Arias, *Structure of multi-component/multi-Yukawa mixtures*, arXiv:cond-mat/0602477 (2006).

- [7] N. J. Wagner, R. Krause, A. R. Rennie, B. D'Aguzzo and J. Goodwin, *The microstructure of polydisperse, charged colloidal suspensions by light and neutron scattering*, J. Chem. Phys. **95**, 494 (1991).
- [8] S. R. Rastogi, N. J. Wagner and S. R. Lustig, *Rheology, self-diffusion, and microstructure of charged colloids under simple shear by massively parallel dynamic simulations*, J. Chem. Phys. **104**, 9249 (1996).
- [9] L. L. Lee, *An accurate integral equation theory for hard spheres: role of the zero-separation theorems in the closure relation*, J. Chem. Phys. **103**, 9388 (1995).